

Network Science Data II

PHYS 7332, Fall 2025

Luca Santagata
Assignment 3

1 *Community detection, degree heterogeneity, and graph-tool (45 points).*

- (a) Recall in `class_09`, we looked at modularity in random (i.e., $G(n, m)$) graphs. Expand this result by replicating Figure 2 from [Guimerà, R., Sales-Pardo, M., & Amaral, L.A.N. \(2004\). *Modularity from fluctuations in random graphs and complex networks. Physical Review E*, 70\(2\), 025101.](#) Comment on this result, especially as it relates to modularity as a measure of partition quality.
- [Figure 1](#) shows the modularity obtained by applying the Louvain algorithm to Barabási–Albert (BA) networks of different sizes S and different values of m , together with the theoretical prediction reported in Eq. (21) of [Guimera et al. \[2004\]](#). Even though the BA model contains no planted community structure, the measured modularity is clearly non-zero for all values of m and grows systematically with the network size. For each fixed m , the empirical modularity increases with S and appears to approach a finite asymptotic value, in agreement with the behavior reported in Figure 2 of [Guimera et al. \[2004\]](#). This is a direct consequence of stochastic fluctuations in the arrangement of edges: even in purely random or scale-free networks, there exist partitions that artificially maximize the density of intra-group edges. Moreover, the effect of m is also consistent with the theory: as m increases, the modularity decreases. Larger m produces denser networks with weaker fluctuations in local connectivity, reducing the ability of modularity maximization algorithms to identify spurious high-modularity partitions. Taken together, these observations make it clear that modularity on its own is not a reliable indicator of meaningful communities. Even networks with no underlying structure can display high modularity purely due to random fluctuations, particularly when they are sparse or highly heterogeneous. For this reason, modularity scores must always be compared against an appropriate null model, rather than being interpreted as direct evidence of genuine community organization.
- (b) Now, using `graph-tool`, fit a degree-corrected and non-degree-corrected stochastic block model to the largest connected component of the `polblogs` dataset, and enforce

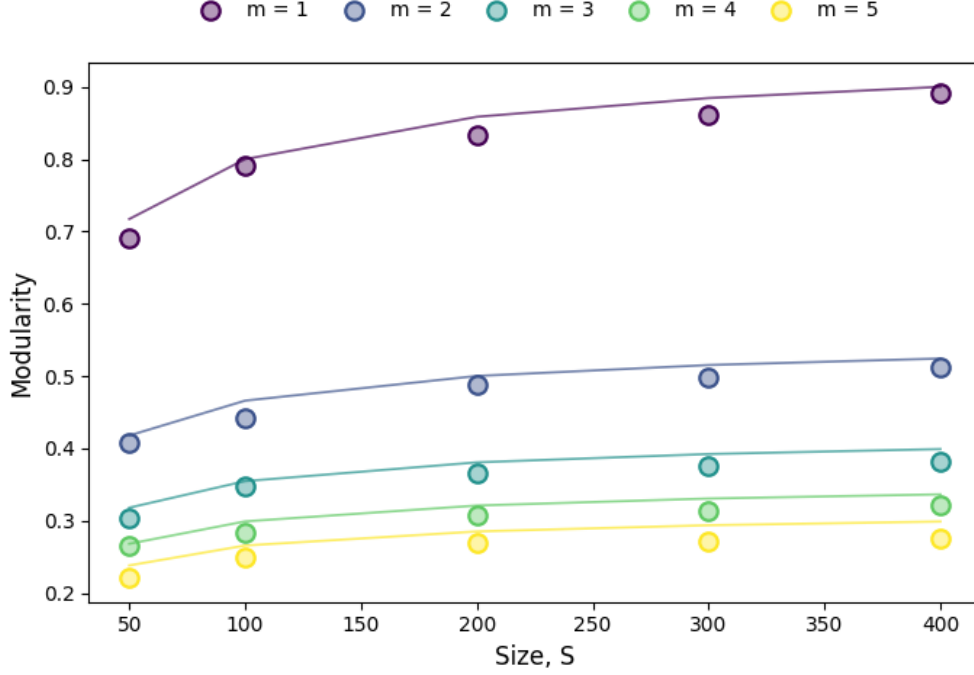


Figure 1: Modularity of scale-free networks generated with the Barabási–Albert model for different values of m and network sizes S .

that the resulting partitions only contain two blocks. Visualize the graph (twice) with the two different partitions. *Hints:* (1) Access this dataset using Netzsleuder by calling `gt.collections.ns["polblogs"]`, (2) You can then run `g = gt.extract_largest_component(g, directed=False)`, and (3) the function `minimize_blockmodel_dl` takes multiple keyword arguments, which you will need to specify to complete this question.

- The result is shown in [Figure 3](#).

(c) Let's dive more into the Markov Chain Monte Carlo (MCMC) algorithm used to arrive at the partitions when fitting stochastic block models.

i) Print the description length of the degree-corrected and non-degree-corrected models you fit in part (b).

- [Table 1](#) highlights a clear difference between the two fitted models. The degree-corrected SBM achieves a substantially lower description length compared to the non-degree-corrected model, indicating that it provides a far more efficient representation of the `polblogs` network. This is expected, as the network exhibits strong degree heterogeneity; without degree correction, the model must rely on a more complex description to account for high-degree nodes, leading to a significantly worse fit.

A similar pattern appears in the modularity values. The degree-corrected partition yields a high modularity ($Q \approx 0.81$), consistent with the well-known

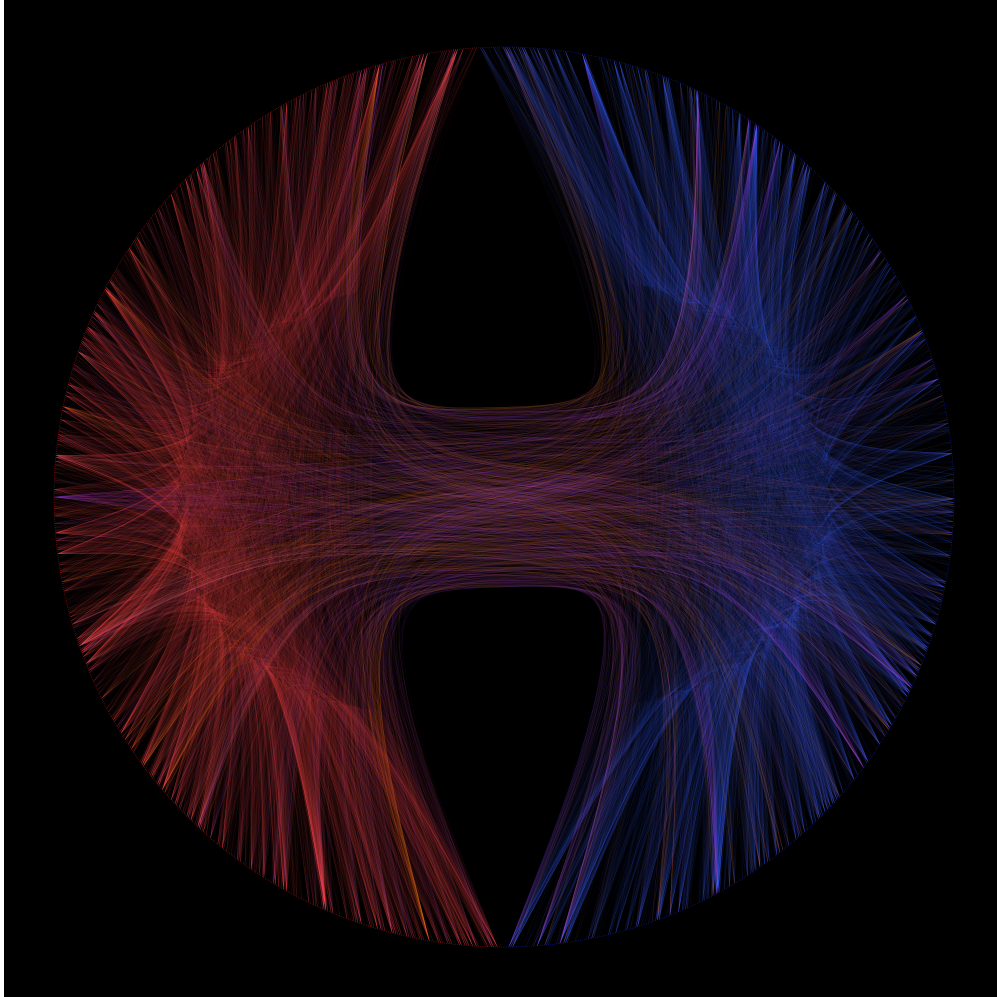
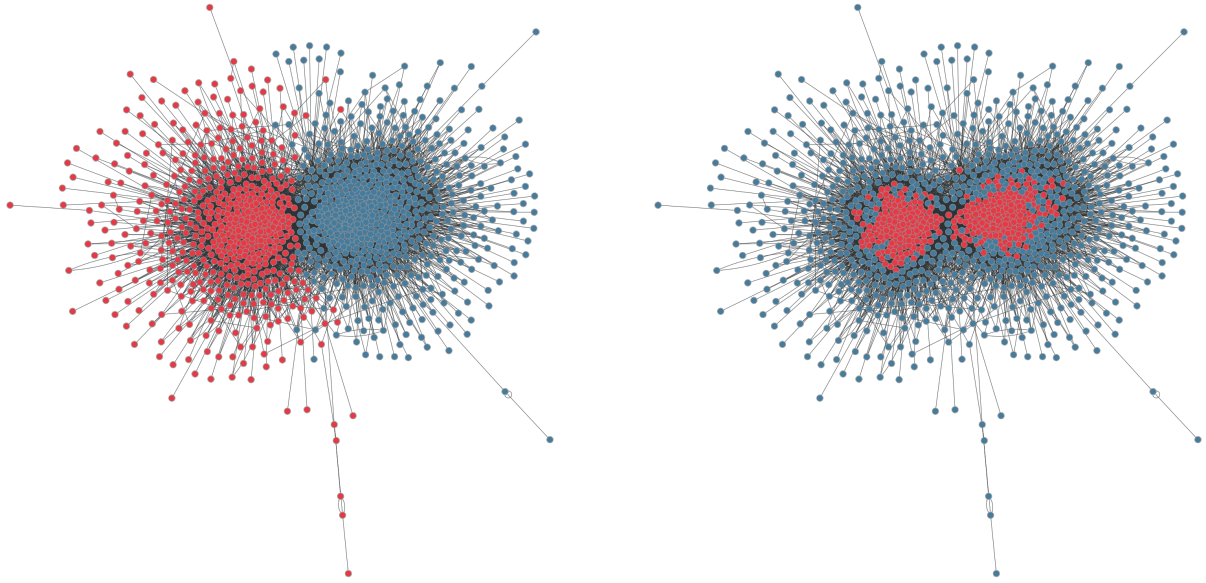


Figure 2: Personal visualization of the largest connected component of the `polblogs` network. Nodes represent political blogs from the 2004 U.S. election, and edges correspond to hyperlinks.

political polarization of the network into two major communities (liberal and conservative). In contrast, the non-degree-corrected model produces a much lower modularity ($Q \approx 0.47$), indicating that its inferred partition does not capture this underlying bipartite structure as effectively. Taken together, these results show that the degree-corrected SBM provides a much more accurate and meaningful two-block representation of the `polblogs` network, while the non-degree-corrected model fails to recover the strong community separation when constrained to only two blocks.

- ii) Fit new degree-corrected and non-degree-corrected models to the largest component of the `polblogs` dataset. After initializing `BlockState` with $B = 2$, use a `for`-loop of 1,000 iterations to collect the description lengths of each model at each step using `mcmc_sweep` (with `niter=1`; during the loop *do not enforce* the number of blocks to remain 2). Visualize both curves in a single, labeled plot and



(a) Degree-corrected SBM

(b) Non-degree-corrected SBM

Figure 3: Two-block partitions inferred from the largest connected component of the `polblogs` network using the degree-corrected and non-degree-corrected stochastic block models.

Model	Description length	Modularity
Degree-corrected SBM	63 428.953	0.8073
Non-degree-corrected SBM	83 166.299	0.4718

Table 1: Comparison of description length and modularity for the degree-corrected and non-degree-corrected SBM fitted to the `polblogs` network (forced $B = 2$).

comment on your observations.

- [Figure 4](#) shows that the two models behave quite differently during the MCMC procedure. The description length of the non-degree-corrected SBM drops quickly at the beginning but soon levels off at a value that is much higher than the degree-corrected model. In contrast, the degree-corrected SBM converges more smoothly and remains consistently lower throughout all 1,000 iterations. This pattern shows that degree correction plays an important role for the `polblogs` network. Because the network has strong degree heterogeneity, the standard SBM struggles to account for high-degree nodes and therefore requires a more complex description. The degree-corrected model, on the other hand, can explain these differences directly, which leads to shorter description lengths and a better overall fit. Overall, the persistent gap between the two curves indicates that the degree-corrected SBM captures the structure of the network more effectively.

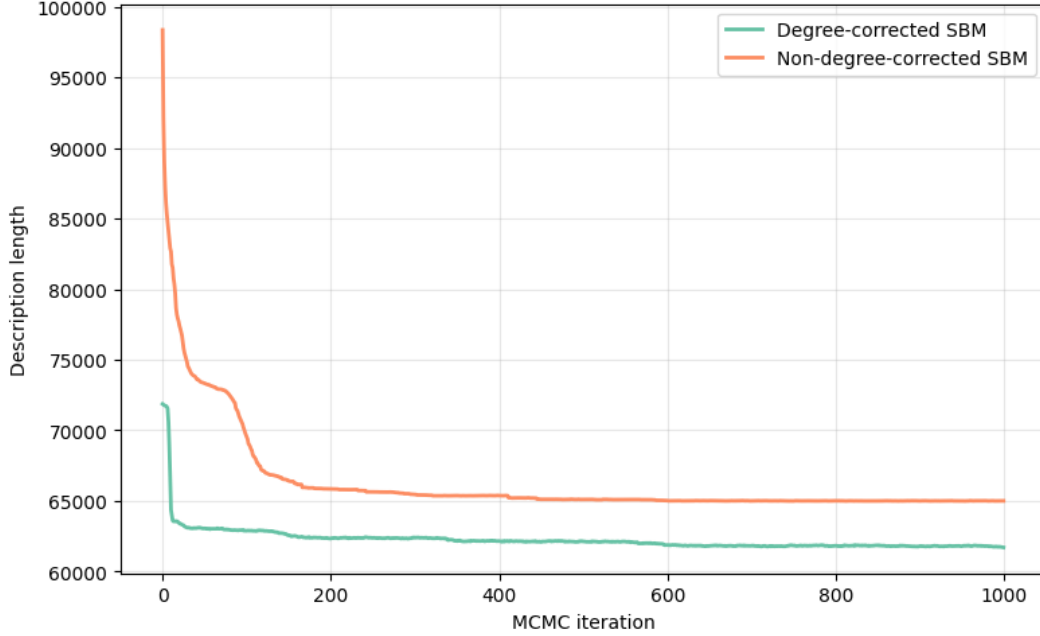


Figure 4: Evolution of the description length during 1,000 MCMC sweeps for the degree-corrected and non-degree-corrected stochastic block models fitted to the largest connected component of the `polblogs` dataset. Both models were initialized with $B = 2$, and at each iteration the description length was recorded using `mcmc_sweep` with `niter=1`, without enforcing the number of blocks to remain fixed.

- (d) Explore the [model selection](#) section in the [graph-tool documentation](#) and Section VIIA and VIIB in <https://arxiv.org/abs/1705.10225>. Figure 7 in that article should validate whether your approach in (b) is on the right track. Summarize the intuition and the formalism behind degree correction in the microcanonical stochastic block model framework. Use Figure 8 as your guide.

- In the non-degree-corrected microcanonical stochastic block model, the N nodes are assigned to blocks via a partition $\mathbf{b} = \{b_i\}$, and the large-scale structure is encoded by the matrix of edge counts $\mathbf{e} = \{e_{rs}\}$, where e_{rs} is the number of edges between blocks r and s . Conditioned on $(\mathbf{b}, \mathbf{e})$, all simple graphs that realize exactly these edge counts are assumed to be equally likely. The likelihood $P(\mathbf{A} \mid \mathbf{b}, \mathbf{e})$ is therefore the inverse of the number of such graphs, and the “entropy” (or description length) used in `graph-tool` is the negative log-likelihood, augmented by priors over $\mathbf{b}$ and $\mathbf{e}$. An important consequence of this formulation is that, inside each block, vertices are statistically equivalent and are expected to have similar degrees. For networks with strong degree heterogeneity (hubs, heavy tails), this assumption is often violated, and the non-degree-corrected SBM tends to group vertices by degree rather than by genuine connectivity patterns. The degree-corrected microcanonical SBM addresses this limitation by adding the degree sequence $\mathbf{k} = \{k_i\}$ as an explicit set of model parameters. In this variant, the ensemble of graphs is constrained not only by the block-to-block edge counts

$\mathbf{e}$, but also by the individual degrees: every sampled graph must have $\deg(i) = k_i$ for all i , while preserving the prescribed connections between blocks. Combinatorially, this is equivalent to a configuration model inside and between blocks: each vertex i is assigned k_i half-edges, and these stubs are paired so as to yield exactly e_{rs} edges between blocks r and s . The microcanonical likelihood is again the inverse of the number of such stub matchings, which can be written in closed form in terms of factorials of $\{e_{rs}\}$ and of the degree multiplicities inside each block. Formally, the parameter set becomes $\boldsymbol{\theta} = \{\mathbf{e}, \mathbf{b}, \mathbf{k}\}$, and the entropy used by `BlockState` with `deg_corr=True` is

$$\Sigma = -\ln P(\mathbf{A}, \boldsymbol{\theta}) = -\ln P(\mathbf{A} \mid \boldsymbol{\theta}) - \ln P(\boldsymbol{\theta}),$$

where the additional term involving $\mathbf{k}$ increases the effective model complexity. Within this Bayesian microcanonical framework, `graph-tool` performs model selection by comparing description lengths across different model variants (e.g. degree-corrected vs. non-degree-corrected, different numbers of blocks, nested vs. non-nested). A more complex model such as the degree-corrected SBM is preferred only if the reduction in $-\ln P(\mathbf{A} \mid \boldsymbol{\theta})$ is large enough to compensate for the extra parameters encoded in $\mathbf{k}$.

2 *Core concepts in machine learning and/or network science. (25 points).*

Learning machine learning and network science can be tough, and sometimes visualizations can help build intuition (either the process of making them, or the process of walking through one). This question asks you to build a *pedagogically effective visualization* of a concept in machine learning or network science.

- (a) Pick a machine learning algorithm or network science technique that was not covered in class. In your words, summarize how it works and what it's useful for.
 - The technique I chose is the *k-core decomposition*, a structural analysis method used to reveal the internal organization and robustness of complex networks. A *k-core* is defined as the largest subgraph in which every node has at least k connections to other nodes within that subgraph. The idea is to iteratively peel off the less connected nodes, those with degree smaller than k , until only a densely connected core remains. The process can be repeated for increasing values of k , revealing nested layers of connectivity from the periphery (low k) to the central core (high k). Each node receives a *core number*, which indicates the deepest layer it belongs to. Nodes in the highest- k core form the structural backbone of the network: they are highly interconnected and tend to hold the network together under perturbations or random failures. This technique is especially useful for studying network robustness and identifying influential nodes or stable communities. For example, in a social network, high-core nodes might correspond to

strongly embedded individuals who are difficult to isolate; in transportation networks, they often represent critical hubs that ensure global connectivity. What I find interesting about k -core decomposition is that it goes beyond simple degree centrality. Instead of just counting connections, it captures how a node's importance depends on its neighbors' connectivity too, providing a collective measure of resilience within the network.

(b) What aspect of the model is most confusing to you? Explain why it is confusing conceptually and walk us through your own understanding in whatever format works best for you.

- The part of the k -core decomposition that initially confused me the most was the idea that a node's "importance" does not depend only on its own degree, but on a kind of collective constraint involving its neighbors. At first glance, it seems like a simple filtering rule: keep only nodes with degree at least k . But this intuition is misleading, because the degree that matters is the degree a node has within the remaining subgraph. This means that the removal of one node may cause several of its neighbors to fall below the degree threshold, triggering a cascade of removals. I found this conceptually confusing because it is not a static property of a node. A node that looks "important" globally can suddenly disappear at higher core levels simply because the neighbors it depends on have already been removed. The k -core is therefore not just a measure of local connectivity, but a recursive, mutually reinforcing property: a node belongs to a high- k core only if its neighbors also survive the same requirement. To make sense of this recursive structure, I walked through the decomposition step-by-step and visualized what happens as k increases. Seeing whole branches of the network peel away clarified why the k -core is sometimes described as an "onion-like" decomposition: each layer reveals a deeper level of structural cohesion. The flashcard I created (in part (c)) reflects this process visually, showing how the network shrinks as the degree requirement becomes more demanding and only the densest part of the graph remains.

(c) Create a [flashcard](#) for the difficult concept you just walked us through. Use `matplotlib`, `LaTeX`, and any other illustrative techniques that are appropriate.

[Potential 5 bonus points if this is made entirely in Python (e.g., Matplotlib, etc.).]

- The flashcard is shown in [Figure 5](#).

3 *Random walk dynamics. (15 points).*

In *class 16*, we introduced the concept of the mean *first-passage time* of random walkers on a graph. Using [Masuda, N., Porter, M. A., & Lambiotte, R. \(2017\). *Random walks and diffusion on networks. Physics Reports*, 716, 1–58](#) as your guide:

K-CORE DECOMPOSITION

The k -core decomposition reveals the internal layers of connectivity within a network.
It progressively filters the graph, showing at each step only the nodes whose degree is at least k .

In this figure:

- At $k = 0$, the full network is displayed: all nodes and edges are present.
- At $k = 1$, the network excludes only the isolate (degree 0), while all connected nodes remain.
- At $k = 2$, nodes with fewer than two neighbors disappear, leaving a smaller but more cohesive subgraph.
- At $k = 3$, only nodes with degree ≥ 3 within the remaining subgraph persist, the periphery fades away.
- At $k = 4$, the innermost and most connected part (the K5 clique) remains, each node having degree 4 within it.
- At $k = 5$, the network becomes empty: no node has enough neighbors to satisfy the condition.

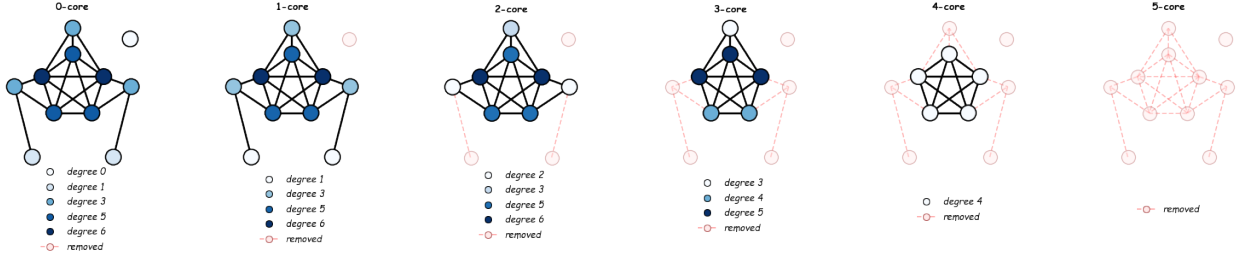


Figure 5: A flashcard visualization illustrating how nodes are progressively removed as the degree threshold k increases. The figure was generated entirely using `Matplotlib`.

- (a) Write a function from scratch that computes the *commute time* of a single random walker between nodes i and j in a network. Test this on the `karate_club` graph by printing the commute time between three randomly selected pairs of nodes.
- The commute time for three randomly selected pairs of nodes is listed in [Table 2](#).

Pair	Nodes (i, j)	Commute Time (mean $\pm \sigma$)
1	(0, 23)	71.77 ± 1.41
2	(12, 8)	128.39 ± 3.27
3	(2, 5)	100.37 ± 2.57

Table 2: Commute time estimates for three randomly selected node pairs in the `karate_club` graph. Each estimate is computed using 100 Monte Carlo samples, with 1000 random-walk trials per sample.

- (b) Calculate the *mean* commute time between pairs of nodes in Erdős–Rényi networks with $N = 500$ and $\langle k \rangle = 8$ and Barabási–Albert networks with $N = 500$ and $m = 4$. Compute the average commute time across all nodes in the network, then average this value across 10 samples from each of these two graph models. Comment on any differences you observe.
- To compute the mean commute time I used the classical relationship between random walks, effective resistance, and the pseudoinverse of the graph Laplacian

([Masuda et al. \[2017\]](#), [Fouss et al. \[2007\]](#)). For an undirected, connected graph G with Laplacian L and Moore–Penrose pseudoinverse L^+ , the commute time between nodes i and j is given by

$$C_{ij} = 2m R_{ij} = 2m(L_{ii}^+ + L_{jj}^+ - 2L_{ij}^+),$$

where m is the number of edges and R_{ij} is the effective resistance distance. I first constructed the combinatorial Laplacian L of the graph and compute its pseudoinverse L^+ using `numpy`’s `pinv` function. Then I built the full matrix of commute times

$$C_{ij} = 2m(L_{ii}^+ + L_{jj}^+ - 2L_{ij}^+)$$

via broadcasting. The mean commute time of the graph is obtained by averaging all entries C_{ij} with $i < j$.

I repeated this computation over:

- 10 Erdős–Rényi (ER) graphs with $N = 500$ and $\langle k \rangle = 8$, extracting the largest connected component when necessary;
- 10 Barabási–Albert (BA) graphs with $N = 500$ and $m = 4$.

The results are:

$$\text{ER: } \overline{C} \approx 1349.87 \pm 26.13, \quad \text{BA: } \overline{C} \approx 1580.62 \pm 6.50.$$

Even though the ER and BA ensembles have the same size N and comparable average degree $\langle k \rangle \approx 8$, the Barabási–Albert networks exhibit systematically higher mean commute times. This difference arises from their distinct structural organization. ER graphs are relatively homogeneous: most nodes have degree close to $\langle k \rangle$, and many paths of similar quality connect arbitrary node pairs. As a result, the effective resistance R_{ij} between nodes is typically small, leading to smaller commute times C_{ij} . BA graphs, on the other hand, are highly heterogeneous due to preferential attachment. A few hubs accumulate very large degree while most nodes remain of low degree. Although hubs shorten shortest-path distances, random walks do not follow geodesics: they diffuse according to local degrees. Low-degree peripheral nodes are effectively separated by bottlenecks at the hubs, and walks can spend a long time circulating around those hubs before reaching the target. From the electrical-network perspective, this structure produces larger effective resistances between many node pairs, hence larger commute times. This interpretation is consistent with the random-walk similarity framework of [Fouss et al. \[2007\]](#), where networks with strong degree heterogeneity or bottlenecks exhibit larger random-walk distances. Finally, the ER ensemble shows a slightly larger standard deviation across samples ($\sigma \approx 26.13$), reflecting variability in edge placement, while the BA ensemble is structurally more constrained and therefore exhibits very low variability between realizations ($\sigma \approx 6.50$).

4 *Meta-question about random walks and spectral properties. (15 points).*

Random walks are one of the simplest ways to model movement or diffusion on a network. In a *local random walk*, a walker moves only to the immediate neighbors of the current node. In contrast, *non-local random walks* allow the walker to “jump” to distant nodes, for example, according to the *fractional graph Laplacian*, which is defined using the eigenvalues and eigenvectors of the standard Laplacian matrix.

- (a) Explain in your own words what the *graph Laplacian* represents and how its eigenvalues and eigenvectors encode information about the connectivity and diffusion properties of a network.
- (b) The *fractional graph Laplacian* L^γ with $0 < \gamma < 1$ generalizes the standard Laplacian. What does lowering γ below 1 change in the behavior of a random walker? Why does this lead to *non-local* or *long-range* movements on the network?
- (c) Using `NetworkX` and `NumPy`, generate a connected undirected network (for example, an Erdős–Rényi or Barabási–Albert graph) with about $N = 300$ nodes. Compute its Laplacian matrix L and its eigen-decomposition. Then create two random-walk transition matrices:

$$\Pi = D^{-1}A, \quad \Pi^{(\gamma)} = I - D_\gamma^{-1}L^\gamma,$$

where D_γ is a diagonal matrix defined from the diagonal elements of L^γ . Simulate both walks for 1000 steps, starting from a random node, and plot the number of distinct nodes visited over time.

- (d) **(bonus points)** Referring to the paper [Random walks on weighted networks: a survey of local and non-local dynamics](#), reproduce Figure 3 in Section 3.5 (*Random walks for image segmentation*) using [Figure 6](#).



Figure 6

References

- Francois Fouss, Alain Pirotte, Jean-Michel Renders, and Marco Saelens. Random-walk computation of similarities between nodes of a graph with application to collaborative recommendation. *IEEE Transactions on knowledge and data engineering*, 19(3):355–369, 2007.
- Roger Guimera, Marta Sales-Pardo, and Luís A Nunes Amaral. Modularity from fluctuations in random graphs and complex networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, 70(2):025101, 2004.
- Naoki Masuda, Mason A Porter, and Renaud Lambiotte. Random walks and diffusion on networks. *Physics reports*, 716:1–58, 2017.